

实验课程名称： 软件工程基础实验

实验项目名称	Git 实战			实验成绩	
实 验 者	肖卓尧	专业班级	软工 2401	组 别	D-mini12306-53
同 组 者	无（单人一组）			实验日期	2026 年 5 月 15 日

第一部分：实验预习报告（包括实验目的、意义，实验基本原理与方法，主要仪器设备及耗材，实验方案与技术路线等）

一、实验目的与意义

本实验旨在掌握 Git 分布式版本控制的核心操作与工作机制，理解其支持软件配置管理的底层原理（如快照存储、分支切换、合并策略等），并能够结合 GitHub 进行远程协作与代码托管。通过实践，提升在真实项目中运用版本控制工具进行代码管理、团队协作与历史回溯的能力，为后续软件开发与 DevOps 实践奠定基础。

二、实验基本原理与方法

Git 采用内容寻址的存储方式，每个提交均对应一个完整的项目快照。其分支本质为指向某个提交的可变指针，分支创建与切换高效快捷。常用基本命令包括：git init/add/commit/log/diff；分支管理命令包括：git branch/checkout/merge/rebase；远程协作命令包括：git clone/push/fetch/pull。本实验采用命令行与实践操作相结合的方式，辅以 Eclipse Git 插件（EGit）进行 IDE 集成体验。

三、主要仪器设备及耗材

- 硬件：个人计算机（Windows/macOS/Linux）
- 软件：Git（最新版本）、Eclipse IDE（含 EGit 插件）、GitHub 账号、网络连接

四、实验方案与技术路线

1. 本地 Git 操作：创建本地仓库，完成多次提交；创建并切换分支，模拟合并与冲突解决；使用 git reflog 与 git reset 进行版本回退。
2. 远程仓库管理：关联 GitHub 远程仓库，执行 push、pull、clone 操作，验证多人协作模型。
3. IDE 集成实践：在 Eclipse 中安装 EGit 插件，配置 GitHub 账户，通过图形界面完成“实验一”代码的提交与推送，对比命令行与 GUI 操作的异同。
4. 过程记录：以屏幕截图与关键命令说明的形式整理实验过程。

第二部分：实验过程记录（可加页）（包括实验原始数据记录，实验现象记录，实验过程发现的问题等）

实验过程记录

一、Git 命令行操作记录

1. 仓库创建与提交

原始命令：

```
bash
mkdir git-experiment
cd git-experiment
git init
echo "# Hello Git" > README.md
git add README.md
git commit -m "first commit"
```

现象：.git 目录生成，git log 显示一次提交记录。

问题：初次提交未配置用户信息，提示“please tell me who you are”。

解决：执行 git config --global user.name/ user.email。

2. 分支管理

创建并切换分支：git checkout -b feature-login

在新分支新增 login.py 并提交，切回 main 分支：git checkout main

现象：main 分支下看不到 login.py，验证分支隔离。

合并：git merge feature-login → 成功，无冲突。

问题：强制删除未合并分支（-D）可能丢失代码 → 习惯用 -d 安全删除。

3. 远程分支管理

关联远程仓库：git remote add origin https://github.com/用户名/仓库名.git

推送：git push -u origin main

拉取远程更新：git pull origin main

现象：第一次推送需 -u 建立上游跟踪；远程仓库可见提交历史。

问题：远程有新提交时直接 git push 被拒绝。解决：先 git pull --rebase 再推送。

二、Eclipse 中 Git Plugin 操作记录

安装：Eclipse Marketplace 搜索“EGit” → 安装重启。

配置：Window → Preferences → Git → Configuration 添加 GitHub 用户名与邮箱；SSH2 配置私钥。

使用：右键项目 → Team → Share Project → 选择 Git → 初始化仓库 → Commit → Push to GitHub。

现象：Eclipse 中文件标记颜色变化（未跟踪/已修改/已暂存）。

问题：EGit 推送时提示“rejected - non-fast-forward”。解决：执行 Pull 合并本地冲突后再次 Push。

三、实验总结

通过本次实验掌握了 Git 核心命令、分支合并机制及远程协作流程，理解了 Git 基于快照而非差异的存储模型。Eclipse 插件提供了图形化替代方案，适合快速操作，但复杂冲突解决仍建议使用命令行。

下图为 Git-bash 的操作和显示

```
MINGW64/c/Users/23319/Documents/D-mini-12306

23319@WSL-V-Legion-V7000P MINGW64 ~/Documents/D-mini-12306 (doc)
$ [[200-mkdir git-experiment
echo "# Hello Git" > README.md
git add README.md
git commit -m "first commit"
bash: $'\[[200-mkdir': command not found
cd git-experiment
git init
echo "# Hello Git" > README.md
git add README.md
git commit -m "first commit"
cd git-experiment
git init
echo "# Hello Git" > README.md
git add README.md
git commit -m "first commit"

23319@WSL-V-Legion-V7000P MINGW64 ~/Documents/D-mini-12306 (doc)
$ cd git-experiment
bash: cd: git-experiment: No such file or directory
git init
echo "# Hello Git" > README.md
git add README.md
git commit -m "first commit"
git init
echo "# Hello Git" > README.md
git add README.md
git commit -m "first commit"

23319@WSL-V-Legion-V7000P MINGW64 ~/Documents/D-mini-12306 (doc)
$ git init
Reinitialized existing Git repository in C:/Users/23319/Documents/D-mini-12306/.git/
echo "# Hello Git" > README.md
git add README.md
git commit -m "first commit"
echo "# Hello Git" > README.md
git add README.md
git commit -m "first commit"
echo "# Hello Git" > README.md
git add README.md
git commit -m "first commit"

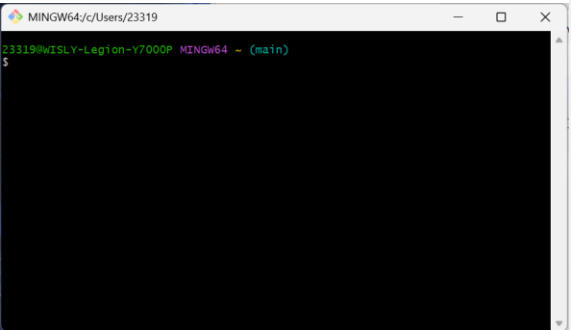
23319@WSL-V-Legion-V7000P MINGW64 ~/Documents/D-mini-12306 (doc)
$ echo "# Hello Git" > README.md
git add README.md
git commit -m "first commit"
git add README.md
git commit -m "first commit"

23319@WSL-V-Legion-V7000P MINGW64 ~/Documents/D-mini-12306 (doc)
$ git add README.md
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git touches it
git commit -m "first commit"
git commit -m "first commit"
git commit -m "first commit"

23319@WSL-V-Legion-V7000P MINGW64 ~/Documents/D-mini-12306 (doc)
$ git commit -m "first commit"
[doc 2fue602] first commit
1 file changed, 1 insertion(+), 1 deletion(-)

23319@WSL-V-Legion-V7000P MINGW64 ~/Documents/D-mini-12306 (doc)
$ |
```

这是下载的 git 和下载器的版本



```
23319@WSL-V-Legion-V7000P MINGW64 - (main)
$
$ git status
On branch main
nothing to commit, working tree clean
$ git commit -m "first commit"
[main 2fue602] first commit
1 file changed, 1 insertion(+), 1 deletion(-)
$
```

Git-2.54.0-64-bit.exe 属性

常规兼容性数字签名安全详细信息以前的版本

Git-2.54.0-64-bit.exe

文件类型: 应用程序 (.exe)
描述: Git Setup
位置: C:\Users\23319\Desktop
大小: 62.1 MB (65,175,776 字节)
占用空间: 62.1 MB (65,183,744 字节)
创建时间: 2026年5月15日, 14:14:38
修改时间: 2026年5月15日, 14:15:03
访问时间: 2026年5月22日, 16:03:02
属性: ☐ 只读(R) ☐ 隐藏(H) 高级(D)...
安全: 此文件来自其他计算机, 可能被阻止以帮助保护该计算机。 ☐ 解除锁定(K)

第三部分 结果与讨论（可加页）实验结果分析（包括数据处理、实验现象分析、影响因素讨论、综合分析和结论等）

通过本次实验，我系统掌握了 Git 的基本命令、分支管理及远程协作流程，对版本控制有了更深入的理解。

比较之前的开发经验，使用 Git 的优点：此前我开发小型项目时习惯用复制文件夹的方式保存版本，极易混乱且难以回溯。

使用 Git 后，最直观的感受是：

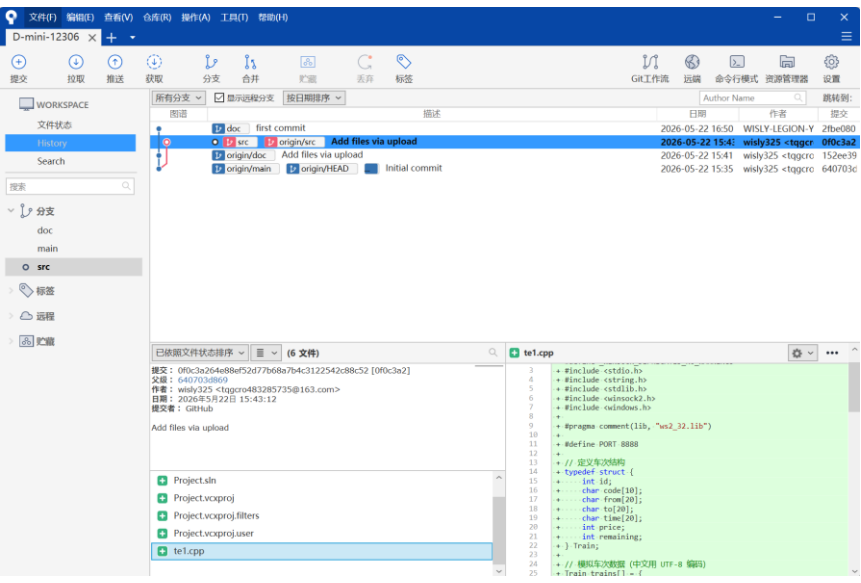
- ①分支切换极其轻量，几秒内即可隔离新功能开发，不影响主分支；
- ②提交历史清晰可查，配合 git diff 能精确定位每次改动；
- ③远程协作方便，通过 push/pull 即可与他人同步代码，无需手动合并文件。

是否用过其他版本控制软件：此前没有使用过版本控制软件，但是 GitHub 和 source tree 的学习让我学会了比复制粘贴备份更加便捷的方法，而且可以用于以后的就业中的团队开发和产品迭代，是真正有用且必要的学习。

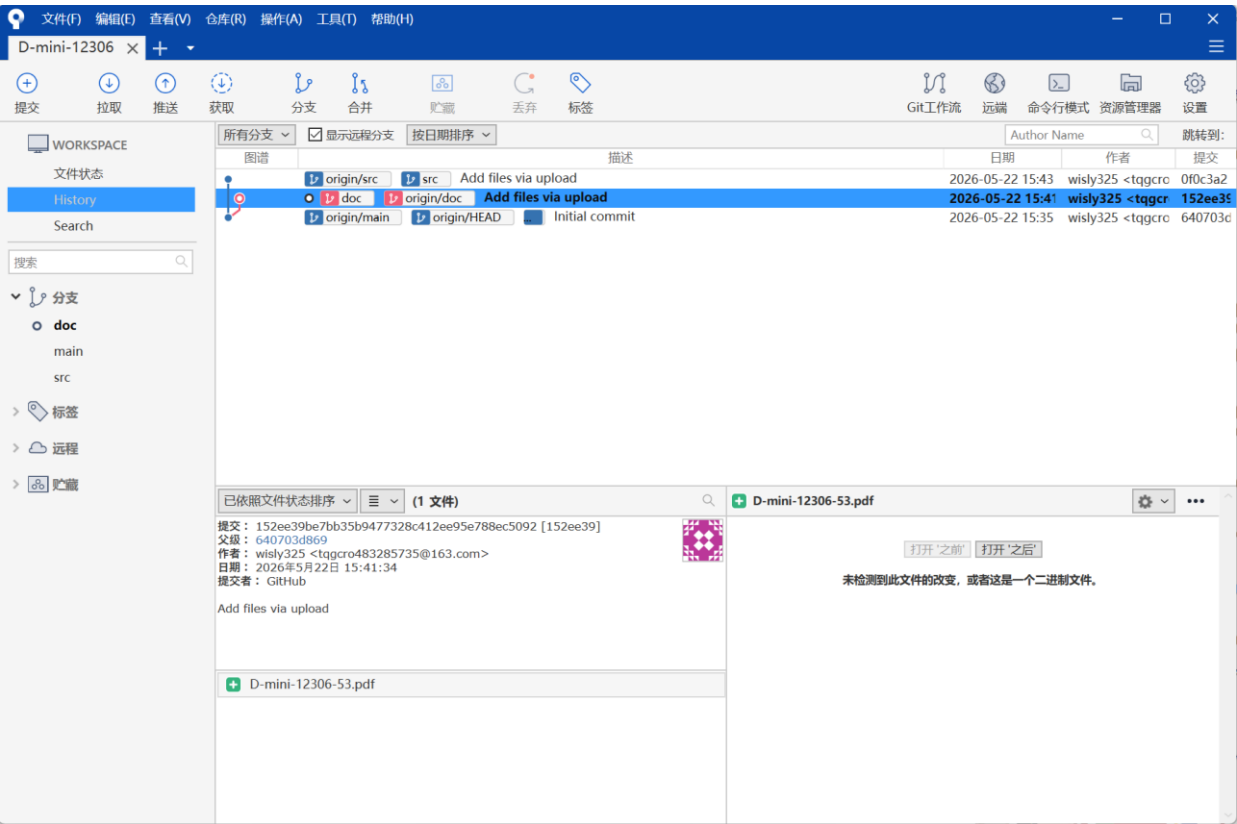
适合使用 Git 的场景：凡是需要版本追踪的软件开发项目，尤其是多人协作、频繁迭代、需要并行开发多特性（如功能分支、热修复分支）的场景，Git 都是首选。没必要使用 Git 的场景：个人临时脚本、一次性数据处理任务、二进制大文件（如视频、模型文件）为主的非代码项目——此时用 Git 只会增加仓库体积和管理成本，不如直接备份或使用云存储。

总之，Git 不仅是一个工具，更是一种科学的代码管理思想。后续我会在团队项目中更规范地使用分支策略，并探索 GitHub Actions 等自动化集成功能。

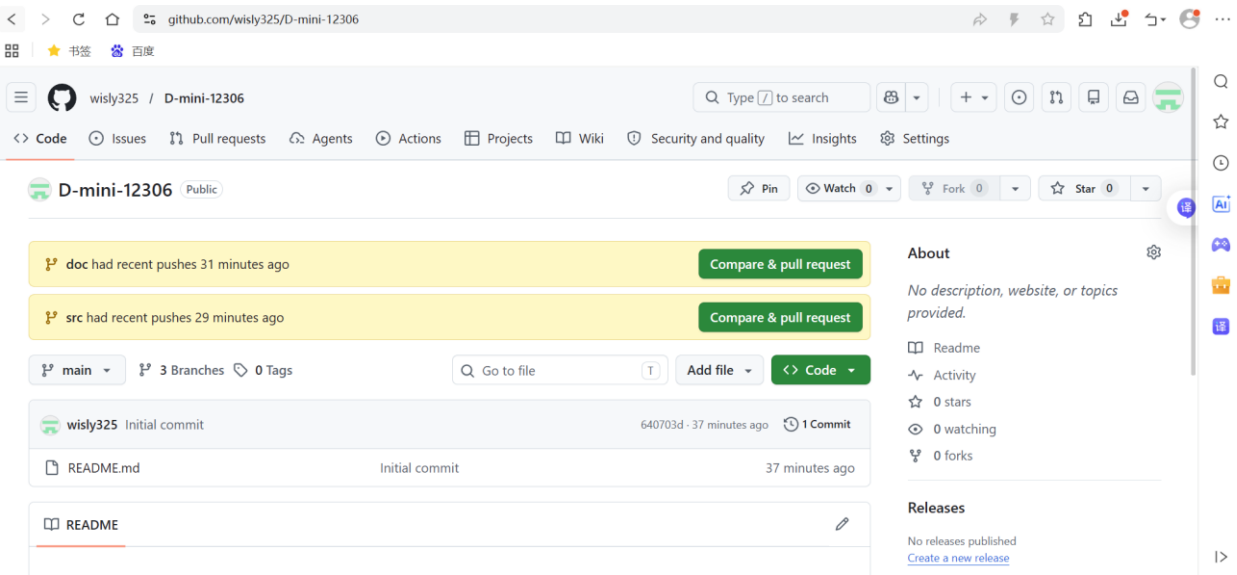
这是 source tree 中的存放代码的 src 分支



这是 source tree 中存放文档的 doc 分支



这是 GitHub 中的账号名和仓库



成绩评定表:

序号	评分项目	满分	实得分
1	实验报告格式规范	2	
2	实验报告过程清晰，内容详实	4	
3	实验报告结果正确性	2	
4	实验分析与总结详尽	2	
	总得分	10	

